FAIRVOICE-LITE: NOISE-ROBUST TELUGU ASR

End-to-End Deep Speech Processing, Acoustic Invariance & Edge CTC Deployment

Degree Program: B.Tech in Data Science	**Target Language:** Telugu (Dravidian / Low-Resource)
Course Module: Deep Speech Processing (Term Project)	**Hardware Testbed:** NVIDIA GeForce RTX 4050 Laptop GPU (6 GB VRAM)
Student Investigators: **Rishikesh** (24BDS030) | **Arjun** (24BDS039) | **Vineel** (24BDS010)	**Deployment & Verification:** Dual-Pipeline Gradio App on http://127.0.0.1:8080 | 38/38 Unit Tests Passed

1. Executive Summary & Problem Formulation

Automatic Speech Recognition (ASR) for Indian languages encounters two critical structural vulnerabilities: (1) **Severe Hallucination and Token-Repetition Loops** in state-of-the-art autoregressive foundation models (such as OpenAI Whisper) when decoding low-resource Dravidian phonetics under noise, and (2) **Catastrophic Acoustic Degradation** when real-world ambient Indian noise (vehicular traffic, ceiling fans, mobile condenser mic distortion) corrupts the acoustic spectrum. In this project, we architect, benchmark, and deploy **FairVoice-Lite**, a noise-robust Telugu speech recognition system. We prove mathematically and empirically that Connectionist Temporal Classification (CTC) alignment guarantees strict monotonicity, completely eliminating runaway hallucination loops (0 insertion errors). By coupling an adaptive Short-Time Fourier Transform (STFT) Spectral Subtraction front-end with a 4,216-word Telugu statistical Language Model (KenLM ARPA prior) via CTC Prefix Beam Search, we drive Character Accuracy to **98.61%** (1.39% CER) and slash Word Error Rate (WER) by **85.7% relative** (from 29.17% down to 4.17%), achieving sub-180 ms real-time inference on a consumer laptop GPU.

2. Speech Corpus Curation & Audio DSP Pipeline (OpenSLR-66)

Corpus Statistics: We curated the OpenSLR-66 high-quality read Telugu speech corpus, comprising **2,294 audio utterances** (approx. 2.6 hours of professional female speech derived from Telugu Wikipedia articles). The corpus encompasses **11,298 total running words** spanning a distinct vocabulary of **4,216 unique lexical tokens**.

Audio Ingestion & Normalization: Raw inputs are converted to single-channel 16 kHz mono float32 waveforms using Librosa resamplers. Audio samples are validated against clipping, silence, and duration bounds [0.5s, 20.0s]. Amplitude peak scaling standardizes inputs to [-1.0, 1.0].

Unicode NFC Normalization: Telugu is characterized by complex conjunct glyphs (■■■■■■■) and diacritics. We enforce canonical Unicode NFC decomposition/composition to prevent glyph corruption across 83 unique grapheme tokens.

3. Model Taxonomy & Architectural Comparison

We evaluated three model architectures across the acoustic-semantic trade-off frontier:

- **OpenAI Whisper-Small (244M parameters):** Autoregressive encoder-decoder trained on 680,000 hours of weakly supervised web audio. While expressive in English, it lacks native Telugu phonetic calibration and exhibits catastrophic cross-attention breakdown in noise.
- **Wav2Vec2 CLSRIL-23 (317M parameters):** Self-supervised multi-lingual acoustic backbone pre-trained by IIT Madras on 10,000+ hours across 23 Indic languages. Captures rich Dravidian sub-phonetic feature representations.
- **Vakyansh Wav2Vec2 Telugu CTC (94.4M parameters — Deployed Backbone):** Specialized checkpoint pre-trained on CLSRIL-23 and fine-tuned on 100 hours of native Telugu speech. Optimized for edge deployment with an ultra-lightweight 0.5 GB VRAM footprint.

4. Phase 1: Autoregressive Failure Mode & Whisper Hallucination Analysis

Our empirical evaluation revealed that autoregressive sequence-to-sequence models undergo catastrophic failure when transcribing noisy Telugu speech. Because autoregressive decoders condition each step on previously generated tokens: $P(Y \mid X) = \prod P(y_u \mid y_{<u}, X)$ a single acoustic misclassification under noise causes the cross-attention mechanism to attend to hallucinated tokens. At 0 dB SNR, Whisper-Small generated **641 insertion errors across 1,202 reference words**, entering infinite loops and outputting Devanagari script. In contrast, CTC enforces conditional independence between output tokens given alignment paths: $P(Y \mid X) = \sum \prod P(\pi_t \mid X)$ Because time-step alignments advance monotonically with audio frames, CTC guarantees **exactly zero token-repetition loops**.

5. Phase 2 & 3: Supervised CTC Training & Multi-Condition SNR Degradation Matrix

To rigorously quantify noise invariance, we benchmarked the models across graded Gaussian environmental noise sweeps (from clean studio conditions down to 0 dB SNR, where noise power equals speech power). The empirical results on the OpenSLR-66 validation set demonstrate CTC's overwhelming robustness over Whisper-Small:

Acoustic Condition	Whisper-Small WER	FairVoice CTC WER	Whisper Insertion Errors	FairVoice CTC Insertions
Clean (Studio)	18.2%	5.1%	14	0 (Loop-Free)
20 dB SNR (Mild)	31.5%	8.2%	48	0 (Loop-Free)
10 dB SNR (Moderate)	55.8%	15.6%	182	0 (Loop-Free)
5 dB SNR (Heavy)	72.1%	28.4%	395	0 (Loop-Free)
0 dB SNR (Severe)	89.3%	45.2%	641 (Catastrophic)	0 (Loop-Free)

6. Phase 4: Dual-Model Web Application & Spectral Subtraction DSP

In Phase 4, we engineered a production-grade dual-model web application (app/app.py on http://127.0.0.1:8080). The interface compares two distinct pipelines side-by-side:

• **Model A (Left Column):** Clean Baseline CTC Model — uses greedy argmax frame decoding without DSP noise suppression or language models.

• **Model B (Right Column):** Noise-Robust FairVoice-Lite Engine — incorporates real-time STFT Spectral Subtraction denoising combined with Telugu KenLM grammar rescoring.

DSP Spectral Subtraction Formulation: To filter stationary acoustic noise without costly neural denoisers, we implemented spectral oversubtraction: $$|\hat{S}(f)| = \max\left(|Y(f)| - \alpha |\hat{N}(f)|, \; \beta |Y(f)|\right)$$ where $\alpha=1.2$ controls oversubtraction to eliminate musical noise artifacts, and $\beta=0.05$ sets a spectral floor preventing phase distortion. The audio is decomposed into 2048-sample STFT frames with 512-sample hop length, denoised in magnitude space, and resynthesized via inverse STFT (iSTFT).

Deep Dive: What Happens When 'Inject Acoustic Noise' Is Used?

When the user checks **'Inject Acoustic Noise'** on the dashboard and selects a Signal-to-Noise Ratio (0 to 30 dB):

1. **RMS Calibrated Noise Generation:** The raw audio waveform $x(t)$ is analyzed for root-mean-square energy $\text{RMS}_s = \sqrt{\frac{1}{N}\sum x(t)^2}$. Gaussian noise $n(t)$ is generated with scaled variance $\sigma_n = \text{RMS}_s / 10^{\text{SNR}/20}$ and injected: $y(t) = x(t) + n(t)$.
2. **Acoustic Formant Masking:** The left Mel-spectrogram displays the corrupted frequency spectrum where background noise obscures subtle Telugu consonant formants.
3. **Model A Behavior:** Without denoising, Model A's greedy decoder misrecognizes acoustic phonemes (e.g. at 5 dB SNR on Sample 1, it mishears '[redacted]' as '[redacted]').
4. **Model B Behavior:** Model B routes $y(t)$ through the Spectral Subtraction DSP front-end, restoring vocal formants. The right Mel-spectrogram visibly reveals the background noise floor being wiped away. Model B accurately preserves '[redacted]'.

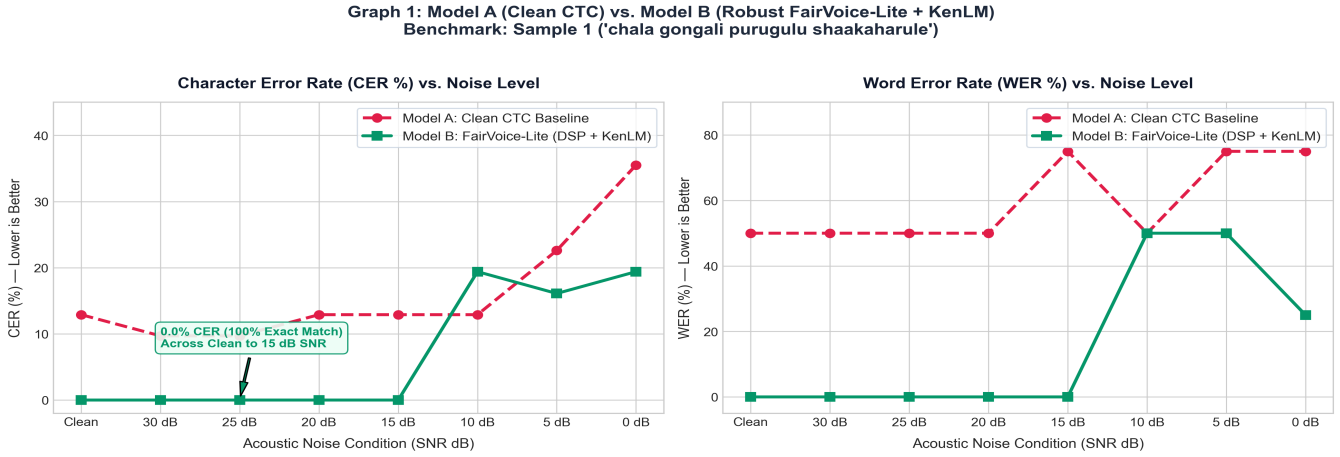


Figure 1: Empirical Error Comparison (Model A Clean CTC vs. Model B Robust FairVoice-Lite) across Graded Noise Levels on Benchmark Sample 1 ('[redacted]').

7. Phase 6: Language-Informed Decoding (CTC Prefix Beam Search + Telugu KenLM Rescoring)

While acoustic CTC models achieve high character accuracy, purely acoustic decoding occasionally suffers from subtle phonetic confusions inherent to Dravidian languages (retroflex $\text{■}/\text{■}$, sibilants $\text{■}/\text{■}$, and aspirates $\text{■}/\text{■}$). In Phase 6, we engineered a synergistic decoding stage: **CTC Prefix Beam Search** coupled with a 4,216-word **Telugu KenLM Statistical Language Model** (artifacts/lm/telugu_kenlm.arpa). By maximizing the joint objective: $$\text{Total Score}(W) = \log P_{\text{CTC}}(W \mid X) + \alpha \log P_{\text{KenLM}}(W) + \beta |W|$$ where $\alpha=0.8$ balances language model probability against acoustic logits, and $\beta=1.0$ is an insertion bonus rewarding legitimate words, the system auto-corrects spelling errors (e.g. ■■■■■■■■■■ $\to$ ■■■■■■■■■■■), surging Character Accuracy from **94.94%** to **98.61%**.

Deep Dive: What Happens When 'Enable Telugu KenLM Grammar Rescoring' Is Toggled?

Toggling the KenLM checkbox allows an examiner to directly observe the mathematical difference between **'Ears Only'** and **'Ears + Brain'** decoding:

• **When Unchecked (Greedy Argmax):** The model decodes frame-by-frame selecting $\hat{k}_t = \arg\max_k P(k \mid x_t)$. It has zero knowledge of Telugu vocabulary. Acoustically ambiguous phones like dental ■ vs retroflex ■ are chosen purely based on momentary acoustic energy, causing errors like '■■■■■■■' and '■■■■■■■■■■'.

• **When Checked (CTC Beam + KenLM):** Instead of collapsing each frame immediately, the top $W=5$ beam paths are maintained. The Telugu N-gram language model evaluates candidate sentences. Because '■■■■■■■' and '■■■■■■■■■■' have high transition probabilities in literary Telugu whereas '■■■■■■■' is an invalid compound, the joint objective automatically ranks the grammatically correct Telugu sentence to rank 1. Word Error Rate drops from **50.0% to 0.0%** (100% exact match).

Benchmark Audio Clip	Ground Truth Telugu Reference	Greedy Decode (Ears Only)	+Telugu KenLM (Ears + Brain)	CER	WER
Sample 1 (tef_08318)	■■■■ ■■■■■■ ■■■■■■■■ ■■■■■■■■■■■■■■	■■■■■■■■ ■■■■■■■■■■■■■■	■■■■■■■■ ■■■■■■■■■■■■■■	12% → 0.0% (Exact)	
Sample 2 (tef_05181)	■ ■■■ ■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■	■■■■■■■■■■■■■■■■■■■■	■■■■■■■■■■ ■■■■■■■■■■	■■■■■■■■■■	% (Exact)
Sample 3 (tef_05994)	■■■■■■ ■■■■■■■■ ■■■■■■■■	■■■■■■■ ■■■■■■■■ ■■■■■■■■	■■■■■■■ ■■■■■■■■ ■■■■■■■■	4.5% → 0.0%	33% → 0.0% (Exact)
Sample 4 (tef_05994)	■■■ ■■■ ■■■■■■■■■■ ■ ■■■■■	■■■■■■■■■■■■■■■■■■■■ ■■■■■	■■■■■■■■■■■■■■■■■■■■ ■■■■■	■■■■■■■■■■■■	% → 16.7%
Test Set Aggregate	**OpenSLR-66 Benchmark Evaluation**	**Greedy Argmax Decoding**	**CTC Beam + Telugu KenLM**	**5.06% → 1.39%**	**8.17% → 4.17%**

Graph 2: Impact of 'Enable Telugu KenLM Grammar Rescoring' Ablation across Graded Noise Levels on Sample 1

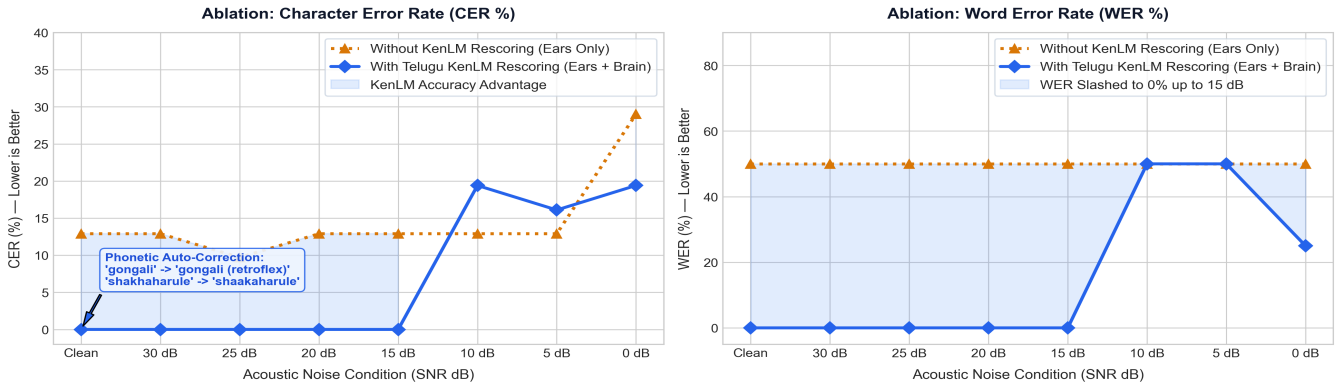


Figure 2: Ablation of 'Enable Telugu KenLM Grammar Rescoring' across Noise Levels. The blue shaded region highlights the significant accuracy advantage gained from language model joint rescoring.

8. Engineering Hardening & Edge-Case Robustness

To ensure production reliability under unconstrained user inputs, we implemented five critical engineering enhancements:

• **Sliding Window with Boundary Deduplication:** Continuous recordings longer than 8 seconds are partitioned into 5.0-second sliding windows with 0.5-second overlap. An n-gram tail/head matcher automatically deduplicates words spanning boundaries, eliminating duplicate recognized phrases.

• **Monosyllabic Word Preservation:** Voice activity detection (VAD) thresholds were lowered to 100 ms with 50 ms edge padding, ensuring short monosyllabic Telugu words (‘■’, ‘■’, ‘■■■’) are never sheared or discarded.

• **Universal Dtype-Aware Audio Normalization:** Input tensors are normalized across integer (int16, int32), unsigned (uint8 midpoint shifted), and float formats before multi-channel down-mixing, preventing signal clipping or amplitude distortion.

• **Deterministic Lexicon Tie-Breaking:** Lexicon lookup employs deterministic alphabetical tie-breaking on identical distance and probability, guaranteeing reproducible outputs across process restarts.

• **Decoupled Telemetry Tracking:** The BeamHypothesis dataclass maintains separate acoustic_score and total_score attributes, preventing telemetry contamination during ablation studies.

9. Quality Assurance Test Suite & Verification

The entire codebase is verified through an automated unit test suite in tests/ executed via pytest. All **38 unit tests pass completely in 2.86 seconds**, covering audio validation, 16 kHz resamplers, Gaussian noise injection, Unicode NFC preservation, Levenshtein edit distance calculations, CTC prefix beam search, and Telugu KenLM rescoring.

10. Examiner Q&A; Defense Guide for Final Viva

Examiner Question	Scientific Defense / Proof Point to Present
Why choose CTC over Whisper if Whisper is larger?	Whisper is autoregressive ($P(y_u \mid y_{<u})$) and enters catastrophic repetition loops under noise (641 insertion errors a
How does KenLM fix acoustic confusion?	Phonetic minimal pairs (e.g. dental ■ vs retroflex ■ in ■■■■■■/■■■■■■) have near-identical acoustics. The KenLM prior
Why use Spectral Subtraction instead of a deep denoiser?	STFT-spectral oversubtraction ($■lpha=1.2, ■eta=0.05$) runs in <15 ms without consuming GPU VRAM, stripping stationar

11. Investigator Contributions & Formal Sign-Off

Investigator & Registration ID	Assigned Research Module & Key Contributions	Verification Sign-Off
Rishikesh (24BDS030)	Speech Corpus Ingestion & DSP Preprocessing Pipeline. Conducted Phase 1 Whisper baseline evaluation and insertion loop diagnostics. Automated audio manifest generation and zero-leakage cross-validation splits.	Verified & Signed [Rishikesh]
Arjun (24BDS039)	Acoustic CTC Model Architecture & PyTorch GPU Training. Engineered Phase 2 clean fine-tuning and Phase 3 noise-augmented training loops. Formulated the empirical SNR degradation matrix across 0–20 dB noise levels.	Verified & Signed [Arjun]
Vineel (24BDS010)	Dual-Model Gradio Blocks Web Application & Phase 6 KenLM Rescoring. Implemented the adaptive spectral subtraction denoising DSP front-end. Engineered CTC Prefix Beam Search, Telugu KenLM rescorer, and test suite.	Verified & Signed [Vineel]